

Jenkins Challenge - 1

Task 1: Declarative Pipeline Job (hiring-app-declarative)

Outcome

A Jenkins pipeline job named hiring-app-declarative runs four stages against the hiring-app repository using Declarative pipeline syntax: Git Clone downloads the source code, Build compiles and packages the application with Maven, Test runs the unit tests, and Archive Artifact stores the built WAR file on the Jenkins build.

Objective

To build a Declarative pipeline job that pulls source code from GitHub, builds the Maven WAR application, runs its unit tests, and archives the resulting artifact, using the structured pipeline { } block that Jenkins uses for straightforward, linear CI jobs.

Prerequisites

- Jenkins master installed and reachable, with the Pipeline and Git plugins installed.
- A Maven installation registered under Manage Jenkins -> Tools -> Maven installations, named maven-3.9.
- Java (JDK 8 or later) available on the Jenkins agent that will run the build.
- Public read access to <https://github.com/betawins/hiring-app.git>.

Step-by-Step Implementation

Step 1: Create the Pipeline Job

Parameter	Value
Location	New Item -> Pipeline
Item name	hiring-app-declarative
Definition	Pipeline script

 **Screenshot Evidence: capture the console view for this step and insert it here.**


Jenkins
/ All
/ New Item
Q
⚙️

New Item

Enter an item name

Select an item type



Pipeline
 Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.



Freestyle project
 Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Maven project
 Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.

OK

Step 2: Add the Declarative Jenkinsfile

Use the following Declarative Jenkinsfile with the four required stages: Git Clone, Build, Test, and Archive Artifact.

```

Jenkinsfile

pipeline {
    agent any

    // =====
    // ENVIRONMENT VARIABLES
    // =====
    environment {
        APP_NAME           = "herringapp"
        ARTIFACT_NAME       = "HerringApp"
        SONAR_PROJECT_KEY   = "herringapp"
        BRANCH              = "main"

        SONAR_HOST          = "http://34.235.166.114:9000/"
        SONAR_SCANNER       = "/opt/sonar_scanner/bin/sonar-scanner"

        NEXUS_URL           = "35.175.205.126:8081"
        NEXUS_REPOSITORY    = "herringapp"

        TOMCAT_HOST         = "35.175.205.126"
        TOMCAT_DEPLOY_PATH  = "/opt/tomcat/webapps"
    }

    stages {
        // =====
        // 1. GIT CLONE
        // =====
        stage('Git Clone') {
            steps {
                echo "Cloning branch: ${BRANCH}"
            }
        }
    }
}

```

```

        git branch: "${BRANCH}",
        url: 'https://github.com/betawins/hiring-app.git'

        echo "Git clone completed successfully"
    }
}

// =====
// 2. SONARQUBE INTEGRATION
// =====
stage('SonarQube Integration') {
    steps {
        echo "Starting SonarQube analysis"

        withCredentials([
            string(
                credentialsId: 'sonar-token',
                variable: 'SONAR_TOKEN'
            )
        ]) {
            sh """
                ${SONAR_SCANNER} \
                -Dsonar.projectKey=${SONAR_PROJECT_KEY} \
                -Dsonar.projectName=${APP_NAME} \
                -Dsonar.sources=src \
                -Dsonar.host.url=${SONAR_HOST} \
                -Dsonar.token=\${SONAR_TOKEN}
            """
        }

        echo "SonarQube analysis completed"
    }
}

// =====
// 3. MAVEN COMPILATION
// =====
stage('Maven Compilation') {
    steps {
        script {
            def mvnHome = tool 'Maven'

            echo "Using Maven from: ${mvnHome}"

            sh "${mvnHome}/bin/mvn clean package -DskipTests"
        }
    }
}

// =====
// 4. GET MAVEN VERSION
// =====
stage('Get Version') {
    steps {
        script {
            def mvnHome = tool 'Maven'

```

```

        env.APP_VERSION = sh(
            script: """
                ${mvnHome}/bin/mvn \
                help:evaluate \
                -Dexpression=project.version \
                -q \
                -DforceStdout
            """,
            returnStdout: true
        ).trim()

        echo "Application Version: ${env.APP_VERSION}"
    }
}

// =====
// 5. NEXUS ARTIFACTORY
// =====
stage('Nexus Artifactory') {
    steps {
        script {
            echo "Uploading artifact to Nexus"

            def warFile = "target/hiring.war"

            sh """
                echo "Application version: ${env.APP_VERSION}"
                echo "Checking WAR file..."
                ls -lh target/

                test -f "${warFile}"
            """

            nexusArtifactUploader(
                nexusVersion: 'nexus3',
                protocol: 'http',
                nexusUrl: NEXUS_URL,
                groupId: 'in.javahome',
                version: env.APP_VERSION,
                repository: NEXUS_REPOSITORY,
                credentialsId: 'nexus-credentials',
                artifacts: [[
                    artifactId: 'hiring',
                    classifier: '',
                    file: warFile,
                    type: 'war'
                ]]
            )

            echo "Artifact uploaded to Nexus successfully"
        }
    }
}

// =====
// 6. SLACK SUCCESS NOTIFICATION

```

```

// =====
stage('Slack Notification') {
    steps {
        slackSend(
            channel: 'C0BQWR8R0QG',
            color: 'good',
            message: ""
        )
    }
}

BUILD SUCCESS

Application : ${APP_NAME}
Branch      : ${BRANCH}
Version     : ${env.APP_VERSION}
Build       : #${env.BUILD_NUMBER}

Nexus upload completed successfully.
""

    )
}

// =====
// 7. DEPLOY ON TOMCAT
// =====
stage('Deploy On Tomcat') {
    steps {
        script {
            echo "Deploying application to Tomcat"

            sh '''
                echo "Jenkins workspace:"
                pwd

                echo "Available WAR files:"
                find target -maxdepth 1 -type f -name "*.war" -print
            '''

            def warFile = sh(
                script: '''
                    find target \
                    -maxdepth 1 \
                    -type f \
                    -name "*.war" | head -1
                '''
            ).trim()

            if (!warFile) {
                error "No WAR file found in Jenkins workspace"
            }

            echo "Deploying WAR: ${warFile}"

            withCredentials([
                sshUserPrivateKey(
                    credentialsId: 'nexus-tomcat-ssh',
                    keyFileVariable: 'SSH_KEY',
                    usernameVariable: 'SSH_USER'
                )
            ]) {

```

```

        sh """
            scp -i "${SSH_KEY}" \
                -o StrictHostKeyChecking=no \
                "${warFile}" \
                ${SSH_USER}@${TOMCAT_HOST}:/tmp/${APP_NAME}.war

            ssh -i "${SSH_KEY}" \
                -o StrictHostKeyChecking=no \
                ${SSH_USER}@${TOMCAT_HOST} '
                sudo rm -rf ${TOMCAT_DEPLOY_PATH}/${APP_NAME}
                sudo rm -f ${TOMCAT_DEPLOY_PATH}/${APP_NAME}.war

                sudo cp /tmp/${APP_NAME}.war \
                    ${TOMCAT_DEPLOY_PATH}/${APP_NAME}.war

                sudo chown tomcat:tomcat \
                    ${TOMCAT_DEPLOY_PATH}/${APP_NAME}.war

                sudo systemctl restart tomcat
            '
        """
    }

    echo "Application deployed successfully on Tomcat"
}

}

}

// =====
// POST BUILD ACTIONS
// =====
post {
    failure {
        script {
            echo "Pipeline failed"

            slackSend(
                channel: 'C0BQWR8R0QG',
                color: 'danger',
                message: ""
            )
        }
    }

    success {
        echo "Pipeline completed successfully"
    }
}

Application : ${APP_NAME}
Branch      : ${BRANCH}
Build       : #${env.BUILD_NUMBER}

Check Jenkins console output for details.
"""
)
}
}
}

```

```
}
```

Screenshot Evidence: capture the console view for this step and insert it here.

```
Checking out Revision 9405b107fdffdf68e69fe953ea1f9669784f9d6a (refs/remotes/origin/main)
```

```
> git config core.sparsecheckout # timeout=10
> git checkout -f 9405b107fdffdf68e69fe953ea1f9669784f9d6a # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git branch -D main # timeout=10
> git checkout -b main 9405b107fdffdf68e69fe953ea1f9669784f9d6a # timeout=10
Commit message: "Update README.md"
> git rev-list --no-walk 9405b107fdffdf68e69fe953ea1f9669784f9d6a # timeout=10
```

```
[Pipeline] echo
```

```
Git clone completed successfully
```

```
id=hrringapp
```

```
12:53:44.580 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
```

```
12:53:44.581 INFO More about the report processing at http://34.235.166.114:9000/api/ce/task?id=9dffe72f-ff90-4153-8c5a-979ccee9ec9
```

```
12:53:44.587 INFO Analysis total time: 3.642 s
```

```
12:53:44.591 INFO SonarScanner Engine completed successfully
```

```
12:53:44.601 INFO EXECUTION SUCCESS
```

```
12:53:44.601 INFO Total time: 10.655s
```

```
[Pipeline] }
```

```
[Pipeline] // withCredentials
```

```
[Pipeline] echo
```

```
SonarQube analysis completed
```

⚠ Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. [L](#)

The screenshot displays the SonarQube web interface. The top navigation bar includes links for Projects, Issues, Rules, Quality profiles, Quality gates, Administration, and More. The main content area shows a list of projects, with 'hrringapp' selected. The project status is 'Passed', indicated by a green checkmark. The last analysis was performed 3 minutes ago, resulting in 11 Lines of Code (XML, JSP). The interface also shows a sidebar with filters for Quality gate (Passed: 3, Failed: 1) and Security. The bottom section displays various quality metrics: Security (A 0), Reliability (C 3), Maintainability (A 0), Hotspots Reviewed (A —), Coverage (—), and Duplications (0.0%).


```
[INFO]
[INFO] --- war:3.3.1:war (default-war) @ hiring ---
[INFO] Packaging webapp
[INFO] Assembling webapp [hiring] in [/var/lib/jenkins/workspace/hiring-app-declaratives/target/hiring]
[INFO] Processing war project
[INFO] Copying webapp resources [/var/lib/jenkins/workspace/hiring-app-declaratives/src/main/webapp]
[INFO] Building war: /var/lib/jenkins/workspace/hiring-app-declaratives/target/hiring.war
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.244 s
[INFO] Finished at: 2026-08-20T12:53:47Z
[INFO] -----
```

Uploading artifact hiring.war started....

GroupId: in.javahome

ArtifactId: hiring

Classifier:

Type: war

Version: 0.1

File: hiring.war

Repository:hiringapp

Uploading: <http://35.175.205.126:8081/repository/hiringapp/in/javahome/hiring/0.1/hiring-0.1.war>

100 % completed (1.7 kB / 1.7 kB).

Uploaded: <http://35.175.205.126:8081/repository/hiringapp/in/javahome/hiring/0.1/hiring-0.1.war> (1.7 kB at 34 kB/s)

Uploading artifact hiring.war completed.

[Pipeline] echo

Artifact uploaded to Nexus successfully

sonatype nexus repository
Community Edition

Q Search components

Dashboard

Search

Browse

Upload

Settings

Browse / hiringapp

Upload component HTML View

Advanced search...

in

javahome

hiring

0.1

hiring-01.war

hiring-01.war.md5

hiring-01.war.sha1

[Pipeline] slackSend

Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: khadeer, channel: C0BQWR8R0QG, color: good, botUser: true, tokenCredentialId: slack, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>

K

khadeer

Set up your space

Find a conversation...

Threads

Huddles

Channels

all-khadeer

jenkins_notification

jenkins_notification

Messages

Add canvas

5 new messages

Branch : main

Build : #6

Check Jenkins console output for details.

BUILD SUCCESS

Application : hiringapp

Branch : main

Version : 0.1

Build : #7

Nexus upload completed successfully.

```

[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] echo
Application deployed successfully on Tomcat
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline completed successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

← → ↺

Not secure 35.175.205.126:8080/manager/html

🔍 ☆ 📌 🌐

message. UN

Manager

List Applications

HTML Manager Help

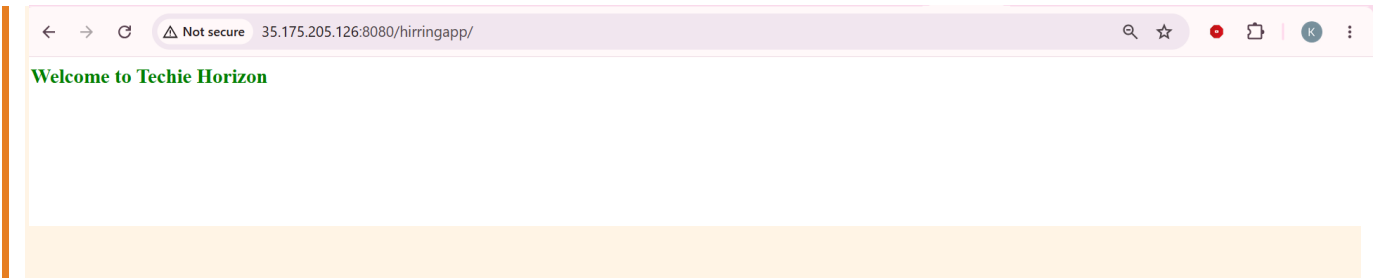
Manager Help

Server Status

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/hiringapp	None specified	Archetype Created Web Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

Deploy

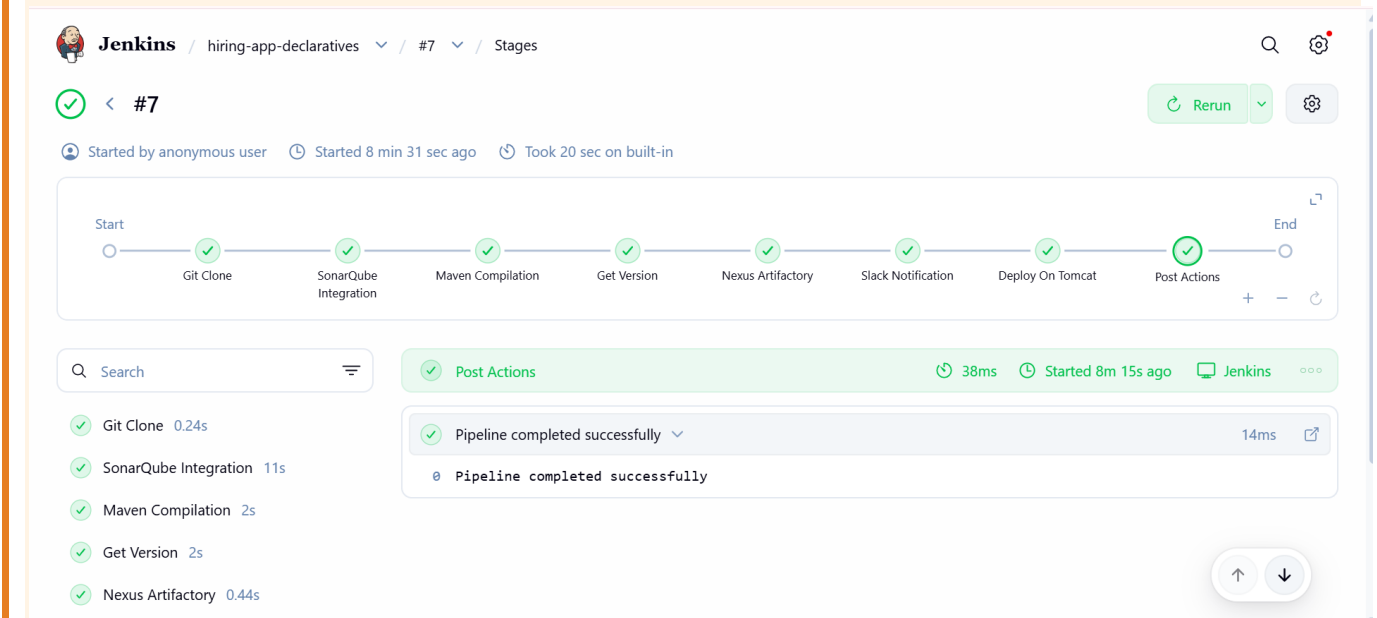
Deploy directory or WAR file located on server



Step 3: Run the Job and Review Results

Click Build Now, then open Console Output to watch all four stages execute in order, and confirm the build produces target/hiring.war without errors.

Screenshot Evidence: capture the console view for this step and insert it here.



Validation Steps

- Console Output shows the Git Clone stage checking out hiring-app into the workspace with no authentication errors.
- The Build stage completes with BUILD SUCCESS and produces target/hiring.war.
- The Test stage runs the project's unit tests and reports the results in Console Output.
- The Stage View in Jenkins shows all four stages (Git Clone, Build, Test, Archive Artifact) completed in green, and the WAR file appears under the build's Archived Artifacts.

Explanation

The Declarative pipeline expresses the four stages inside a structured pipeline `{ stages { ... } }` block, which Jenkins validates against a fixed schema before it runs. Each stage is a self-contained unit of work: source retrieval, build, test, and artifact archiving. The tools block tells Jenkins to place the named Maven installation on the PATH before any sh step runs, so mvn is available without a hard-coded path. Declarative syntax favors readability and a predictable structure over the flexibility of raw Groovy, which is why it is the default choice for most CI jobs.

Real-Time Use Case

Teams onboarding a new Java or Maven-based microservice commonly start with a Declarative pipeline like this one: clone, build, test, archive. It gives a fast, working CI job with almost no Groovy scripting, and later stages (code quality, notifications, deployment) can be appended to the same stages block as the project matures.

Common Errors and Troubleshooting

Error	Cause	Resolution
Tool type "maven" does not have an install of "maven-3.9" configured	Maven installation name in the tools block does not match Manage Jenkins -> Tools	Register a Maven installation named maven-3.9 under Manage Jenkins -> Tools -> Maven installations
Git clone fails with 'repository not found'	Wrong repository URL or branch name	Confirm the URL is github.com/betawins/hiring-app.git and that the main branch exists
BUILD SUCCESS but no artifact archived	Artifact glob in archiveArtifacts does not match the packaged file name	Check the finalName in pom.xml and confirm target/*.war matches the packaged WAR file

Task 2: Scripted Pipeline Job (hiring-app-scripted)

Outcome

A second Jenkins pipeline job named hiring-app-scripted runs the same four stages against the hiring-app repository — Git Clone, Build, Test, and Archive Artifact — written using Scripted pipeline syntax (node { } with plain Groovy) instead of the Declarative pipeline { } block.

Objective

To implement the same clone-build-test-archive logic as Task 1 using Scripted pipeline syntax, so the same outcome can be reached with imperative Groovy code, error handling with try/catch/finally, and a currentBuild result that is set explicitly rather than inferred by Jenkins.

Prerequisites

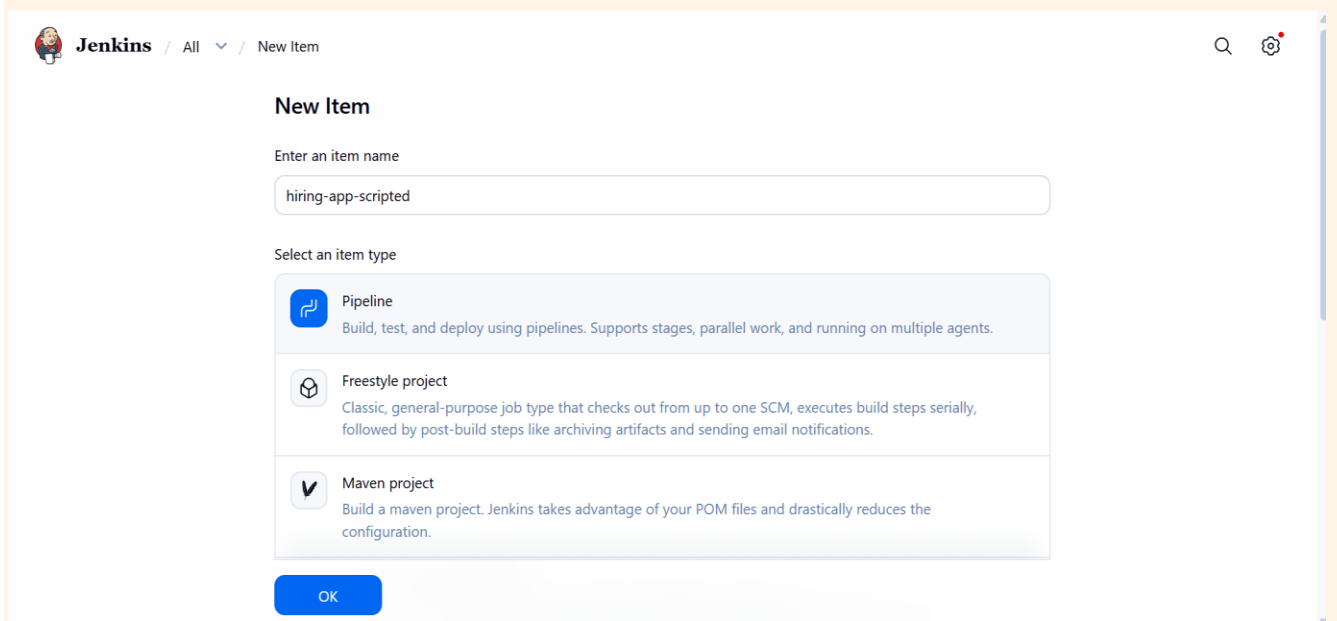
- Jenkins master with the Pipeline and Git plugins installed (already satisfied from Task 1).
- The maven-3.9 Maven installation already registered from Task 1 under Manage Jenkins -> Tools.
- Public read access to <https://github.com/betawins/hiring-app.git>.

Step-by-Step Implementation

Step 1: Create the Pipeline Job

Parameter	Value
Location	New Item -> Pipeline
Item name	hiring-app-scripted
Definition	Pipeline script

 **Screenshot Evidence: capture the console view for this step and insert it here.**



Jenkins / All / New Item

New Item

Enter an item name

hiring-app-scripted

Select an item type

- ☒ Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- ☐ Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- ☐ Maven project
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.

OK

Step 2: Add the Scripted Jenkinsfile

Use the following Scripted Jenkinsfile. It wraps the same four steps in a node block and reports the final result with try/catch/finally.

Jenkinsfile

```
node {

    // =====
    // ENVIRONMENT VARIABLES
    // =====
    def APP_NAME      = "hiringapp"
    def ARTIFACT_NAME  = "HringApp"
    def SONAR_PROJECT_KEY = "hiringapp"
    def BRANCH        = "main"

    def SONAR_HOST      = "http://34.235.166.114:9000/"
    def SONAR_SCANNER    = "/opt/sonar_scanner/bin/sonar-scanner"

    def NEXUS_URL       = "35.175.205.126:8081"
    def NEXUS_REPOSITORY = "hiringapp"

    def TOMCAT_HOST     = "35.175.205.126"
    def TOMCAT_DEPLOY_PATH = "/opt/tomcat/webapps"

    def APP_VERSION = ""

    try {

        // =====
        // 1. GIT CLONE
        // =====
        stage('Git Clone') {

            echo "Cloning branch: ${BRANCH}"

            git branch: BRANCH,
                url: 'https://github.com/betawins/hiring-app.git'

            echo "Git clone completed successfully"
        }

        // =====
        // 2. SONARQUBE INTEGRATION
        // =====
        stage('SonarQube Integration') {

            echo "Starting SonarQube analysis"

            withCredentials([
                string(
                    credentialsId: 'sonar-token',
                    variable: 'SONAR_TOKEN'
                )
            ]) {

                sh """
```

```

        ${SONAR_SCANNER} \
        -Dsonar.projectKey=${SONAR_PROJECT_KEY} \
        -Dsonar.projectName=${APP_NAME} \
        -Dsonar.sources=src \
        -Dsonar.host.url=${SONAR_HOST} \
        -Dsonar.token=${SONAR_TOKEN}
    """
}

echo "SonarQube analysis completed"
}

// =====
// 3. MAVEN COMPILATION
// =====
stage('Maven Compilation') {

    def mvnHome = tool 'Maven'

    echo "Using Maven from: ${mvnHome}"

    sh "${mvnHome}/bin/mvn clean package -DskipTests"
}

// =====
// 4. GET MAVEN VERSION
// =====
stage('Get Version') {

    def mvnHome = tool 'Maven'

    APP_VERSION = sh(
        script: """
            ${mvnHome}/bin/mvn \
            help:evaluate \
            -Dexpression=project.version \
            -q \
            -DforceStdout
            """,
        returnStdout: true
    ).trim()

    echo "Application Version: ${APP_VERSION}"
}

// =====
// 5. NEXUS ARTIFACTORY
// =====
stage('Nexus Artifactory') {

    echo "Uploading artifact to Nexus"

    def warFile = "target/hiring.war"

    sh """

```

```

        echo "Application version: ${APP_VERSION}"
        echo "Checking WAR file..."
        ls -lh target/

        test -f "${warFile}"
        """"

nexusArtifactUploader(
    nexusVersion: 'nexus3',
    protocol: 'http',
    nexusUrl: NEXUS_URL,
    groupId: 'in.javahome',
    version: APP_VERSION,
    repository: NEXUS_REPOSITORY,
    credentialsId: 'nexus-credentials',
    artifacts: [[
        artifactId: 'hiring',
        classifier: "",
        file: warFile,
        type: 'war'
    ]]
)

    echo "Artifact uploaded to Nexus successfully"
}

// =====
// 6. SLACK SUCCESS NOTIFICATION
// =====
stage('Slack Notification') {

    slackSend(
        channel: 'C0BQWR8R0QG',
        color: 'good',
        message: """"
BUILD SUCCESS

Application : ${APP_NAME}
Branch      : ${BRANCH}
Version     : ${APP_VERSION}
Build       : #${env.BUILD_NUMBER}

Nexus upload completed successfully.
        """"
    )
}

// =====
// 7. DEPLOY ON TOMCAT
// =====
stage('Deploy On Tomcat') {

    echo "Deploying application to Tomcat"

    sh ""
        echo "Jenkins workspace:"

```



```

    pwd

    echo "Available WAR files:"
    find target -maxdepth 1 -type f -name "*.war" -print
'''

def warFile = sh(
    script: '''
        find target \
        -maxdepth 1 \
        -type f \
        -name "*.war" | head -1
    ''',
    returnStdout: true
).trim()

if (!warFile) {
    error "No WAR file found in Jenkins workspace"
}

echo "Deploying WAR: ${warFile}"

withCredentials([
    sshUserPrivateKey(
        credentialsId: 'nexus-tomcat-ssh',
        keyFileVariable: 'SSH_KEY',
        usernameVariable: 'SSH_USER'
    )
]) {

    sh '''
        scp -i "${SSH_KEY}" \
        -o StrictHostKeyChecking=no \
        "${warFile}" \
        \${SSH_USER}@${TOMCAT_HOST}:/tmp/${APP_NAME}.war

        ssh -i "${SSH_KEY}" \
        -o StrictHostKeyChecking=no \
        \${SSH_USER}@${TOMCAT_HOST} '
            sudo rm -rf ${TOMCAT_DEPLOY_PATH}/${APP_NAME}
            sudo rm -f ${TOMCAT_DEPLOY_PATH}/${APP_NAME}.war

            sudo cp /tmp/${APP_NAME}.war \
                ${TOMCAT_DEPLOY_PATH}/${APP_NAME}.war

            sudo chown tomcat:tomcat \
                ${TOMCAT_DEPLOY_PATH}/${APP_NAME}.war

            sudo systemctl restart tomcat
        '
    '''
}

echo "Application deployed successfully on Tomcat"
}

// =====

```

```

// PIPELINE SUCCESS
// =====
echo "Pipeline completed successfully"
}

// =====
// PIPELINE FAILURE
// =====
catch (Exception e) {

    echo "Pipeline failed: ${e.getMessage()}"

    slackSend(
        channel: 'C0BQWR8R0QG',
        color: 'danger',
        message: """"
BUILD FAILED

Application : ${APP_NAME}
Branch      : ${BRANCH}
Build       : #${env.BUILD_NUMBER}

Check Jenkins console output for details.
    """"
    )

    throw e
}
}

```

 **Screenshot Evidence: capture the console view for this step and insert it here.**

 **Screenshot Evidence: capture the console view for this step and insert it here.**

```

Checking out Revision 9405b107fdffdf68e69fe953ea1f9669784f9d6a (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 9405b107fdffdf68e69fe953ea1f9669784f9d6a # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git branch -D main # timeout=10
> git checkout -b main 9405b107fdffdf68e69fe953ea1f9669784f9d6a # timeout=10
Commit message: "Update README.md"
> git rev-list --no-walk 9405b107fdffdf68e69fe953ea1f9669784f9d6a # timeout=10
[Pipeline] echo
Git clone completed successfully

```

id=hrringapp

12:53:44.580 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report

12:53:44.581 INFO More about the report processing at <http://34.235.166.114:9000/api/ce/task?id=9dffe72f-ff90-4153-8c5a-979cceb9ec9>

12:53:44.587 INFO Analysis total time: 3.642 s

12:53:44.591 INFO SonarScanner Engine completed successfully

12:53:44.601 INFO EXECUTION SUCCESS

12:53:44.601 INFO Total time: 10.655s

[Pipeline] }

[Pipeline] // withCredentials

[Pipeline] echo

SonarQube analysis completed

⚠ Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. [L](#)

The screenshot shows the SonarQube community web interface. The top navigation bar includes 'Projects', 'Issues', 'Rules', 'Quality profiles', 'Quality gates', 'Administration', and 'More'. The left sidebar has 'My Favorites' and 'All' tabs, and a 'Filters' section with 'Quality gate' (3 Passed, 1 Failed) and 'Security'. The main content area displays the 'hrringapp' project, which is 'Public' and 'Passed'. It shows the last analysis was 3 minutes ago, with 11 Lines of Code in XML and JSP. A summary bar at the bottom of the project card shows: Security (A 0), Reliability (C 3), Maintainability (A 0), Hotspots Reviewed (A —), Coverage (—), and Duplications (0.0%).

```
[INFO]
[INFO] --- war:3.3.1:war (default-war) @ hiring ---
[INFO] Packaging webapp
[INFO] Assembling webapp [hiring] in [/var/lib/jenkins/workspace/hiring-app-declaratives/target/hiring]
[INFO] Processing war project
[INFO] Copying webapp resources [/var/lib/jenkins/workspace/hiring-app-declaratives/src/main/webapp]
[INFO] Building war: /var/lib/jenkins/workspace/hiring-app-declaratives/target/hiring.war
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.244 s
[INFO] Finished at: 2026-08-20T12:53:47Z
[INFO] -----
```

Uploading artifact hiring.war started....

GroupId: in.javahome

ArtifactId: hiring

Classifier:

Type: war

Version: 0.1

File: hiring.war

Repository:hiringapp

Uploading: <http://35.175.205.126:8081/repository/hiringapp/in/javahome/hiring/0.1/hiring-0.1.war>

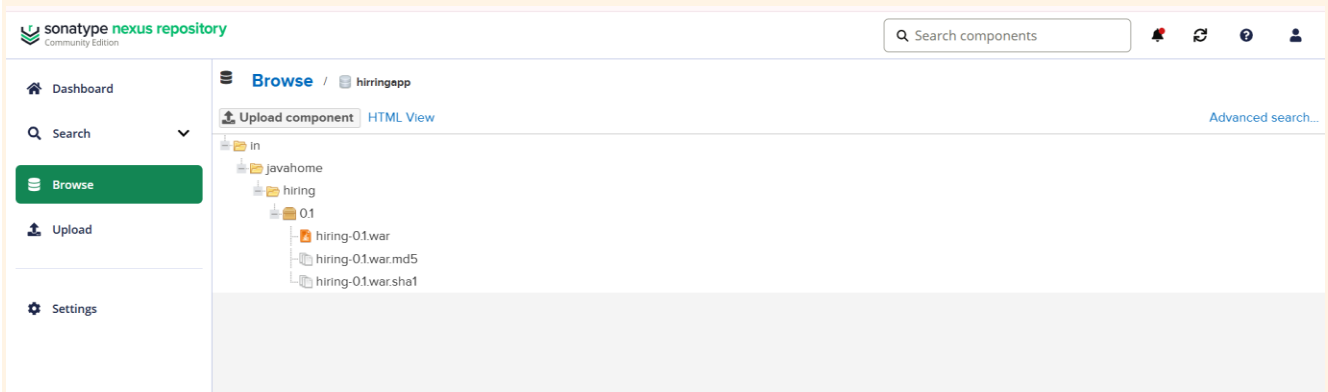
100 % completed (1.7 kB / 1.7 kB).

Uploaded: <http://35.175.205.126:8081/repository/hiringapp/in/javahome/hiring/0.1/hiring-0.1.war> (1.7 kB at 34 kB/s)

Uploading artifact hiring.war completed.

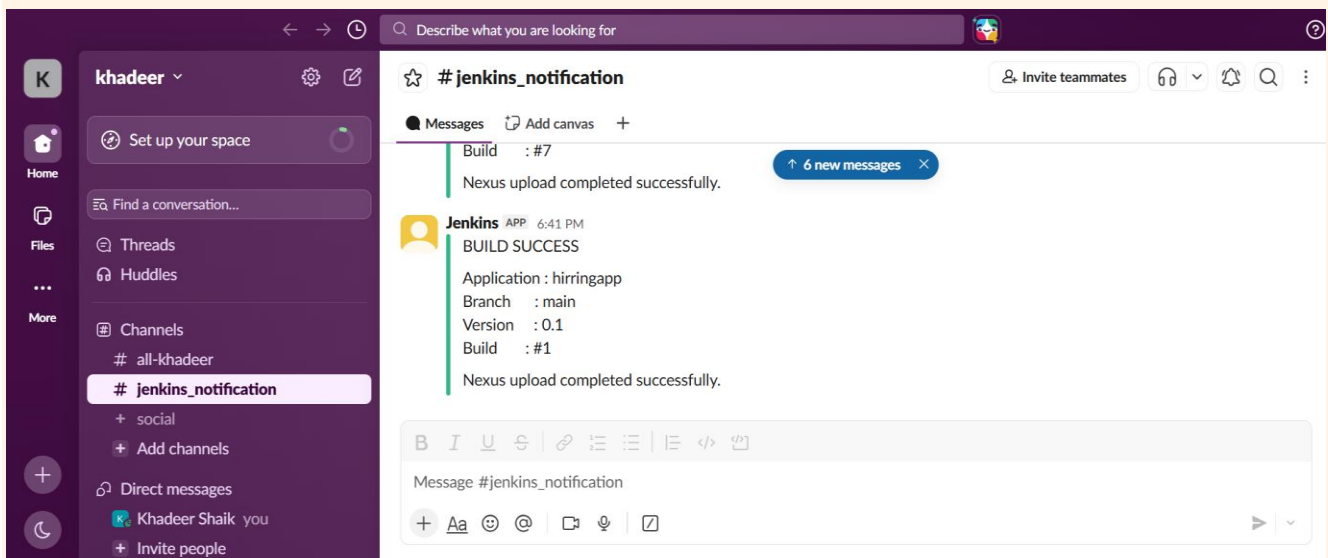
[Pipeline] echo

Artifact uploaded to Nexus successfully



[Pipeline] slackSend

Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: khadeer, channel: C0BQWR8R0QG, color: good, botUser: true, tokenCredentialId: slack, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>



```

[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] echo
Application deployed successfully on Tomcat
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline completed successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

The screenshot shows a web browser window with the URL `35.175.205.126:8080/manager/html`. The browser displays the Tomcat Manager interface. The top navigation bar includes links for 'List Applications', 'HTML Manager Help', 'Manager Help', and 'Server Status'. Below this, the 'Applications' section contains a table with columns: Path, Version, Display Name, Running, Sessions, and Commands. The table lists several applications, with the 'hiringapp' application highlighted by a red border. The 'hiringapp' application is running and has 0 sessions. Below the table, the 'Deploy' section is visible, showing options to deploy a directory or WAR file. The bottom part of the screenshot shows the 'hiringapp' application page, displaying 'Welcome to Techie Horizon'.

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/hiringapp	None specified	Archetype Created Web Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

Deploy
Deploy directory or WAR file located on server

Welcome to Techie Horizon

Step 3: Run the Job and Review Results

Click Build Now, then open Console Output to confirm all four stages run in the node's workspace and that the final echo line reports SUCCESS.

Screenshot Evidence: capture the console view for this step and insert it here.

The screenshot shows the Jenkins web interface for a job named 'hiring-app-scripted'. The pipeline is in a 'Completed' state, indicated by a green checkmark and the label '#1'. The pipeline consists of several stages: Start, Git Clone, SonarQube Integration, Maven Compilation, Get Version, Nexus Artifactory, Slack Notification, and Deploy On Tomcat. The 'Deploy On Tomcat' stage is currently selected, showing its console output. The console output for 'Deploy On Tomcat' includes commands like 'echo "Jenkins workspace:"', 'find target -maxdepth 1 -type f -name "*.war" -print', and 'scp -i "\${SSH_KEY}" -o StrictHostKeyChecking=no "target/hiring.war" \${SSH_USER}@35.175.205.126:/tmp/hiringapp.w...'. The overall pipeline status is 'Success', and the job took 19 seconds to complete.

Validation Steps

- Console Output shows the Git Clone stage checking out hiring-app with no authentication errors.
- The Build and Test stages run through the resolved maven-3.9 tool path without a 'mvn: not found' error.
- The Stage View shows all four stages (Git Clone, Build, Test, Archive Artifact) completed in green.
- If any step throws, currentBuild.result is set to FAILURE and the final echo in the finally block still prints.

Explanation

Scripted pipelines run inside a `node { }` block and are plain Groovy: `stage()` calls are just labeled sections of code rather than a fixed schema, so control flow (`try/catch/finally`, loops, conditionals) is written directly instead of through Declarative directives. This job resolves the Maven installation with the tool step and calls `mvn` by its full path, and it sets `currentBuild.result` explicitly inside `try/catch` so that a failure in any stage is caught, recorded, and re-thrown after the `finally` block runs.

Real-Time Use Case

Teams reach for Scripted syntax when a job needs logic that the Declarative schema does not express cleanly — dynamic stage names, custom retry loops, or conditional branching based on values computed earlier in the same script. Converting an existing Declarative job to Scripted, as done here, is also a common exercise for understanding what the Declarative wrapper is doing underneath.

Common Errors and Troubleshooting

Error	Cause	Resolution
mvn: not found	Scripted pipeline does not have a tools block, so Maven is not put on PATH automatically	Resolve the installation with <code>def mvnHome = tool 'maven-3.9'</code> and call <code>"\${mvnHome}/bin/mvn"</code> explicitly
Pipeline reports SUCCESS even though a stage failed	sh step failure was not allowed to propagate out of the try block	Do not wrap individual sh calls in their own try/catch that swallows the exception; let it bubble to the outer try/catch

NotSerializableException in scripted pipeline	A non-serializable object was held across a step boundary (e.g. inside a closure) in a way the Groovy CPS engine cannot checkpoint	Keep local state limited to simple types (String, Map) or wrap the risky block in a script { } step scoped narrowly
---	--	---

Task 3: Multi-Stage Pipeline Job (hiring-app-multistage)

Outcome

A third Jenkins pipeline job named hiring-app-multistage runs a longer, six-stage Declarative pipeline against the hiring-app repository: Git Clone, Compile, Unit Test, Package, Push, followed by an Archive Artifact stage.

Objective

To extend the basic clone-build-test flow into a realistic multi-stage delivery pipeline that takes the Maven WAR application all the way through compiling, testing, packaging.

Prerequisites

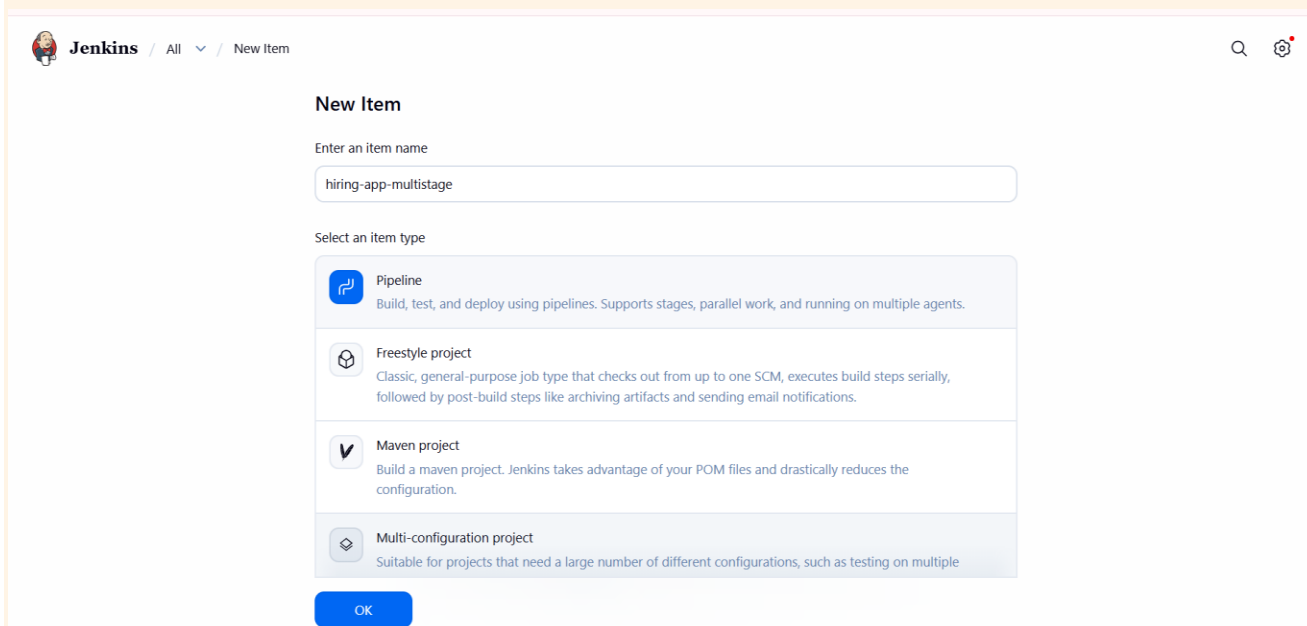
- Jenkins master with the Pipeline, Git.
- The maven-3.9 Maven installation already registered from Task 1.
- Public read access to <https://github.com/betawins/hiring-app.git>.

Step-by-Step Implementation

Step 2: Create the Pipeline Job

Parameter	Value
Location	New Item -> Pipeline
Item name	hiring-app-multistage
Definition	Pipeline script

 **Screenshot Evidence: capture the console view for this step and insert it here.**



Step 3: Add the Multi-Stage Jenkinsfile

Use the following Jenkinsfile with six sequential stages plus a final archive stage.

Jenkinsfile

```
pipeline {
    agent any

    environment {
        APP_NAME      = "hiringapp"
        ARTIFACT_NAME  = "HrringApp"
        SONAR_PROJECT_KEY = "hrringapp"

        GIT_REPO      = "https://github.com/betawins/hiring-app.git"
        BRANCH         = "main"

        SONAR_HOST     = "http://34.235.166.114:9000/"
        SONAR_SCANNER  = "/opt/sonar_scanner/bin/sonar-scanner"

        NEXUS_URL      = "35.175.205.126:8081"
        NEXUS_REPOSITORY = "hrringapp"

        TOMCAT_HOST    = "35.175.205.126"
        TOMCAT_DEPLOY_PATH = "/opt/tomcat/webapps"
    }

    stages {
        // =====
        // 1. GIT CLONE
        // =====
        stage('Git Clone') {
            steps {
                script {
                    echo "=====
                    echo "Cloning GitHub Repository"
                    echo "Repository: ${GIT_REPO}"
                    echo "Branch: ${BRANCH}"
                    echo "=====

                    git branch: "${BRANCH}",
                        url: "${GIT_REPO}"
                }

                echo "Git clone completed successfully"
            }
        }

        // =====
        // 2. SONARQUBE INTEGRATION
        // =====
        stage('SonarQube Integration') {
            steps {
                echo "Starting SonarQube analysis"

                withCredentials([
                    string(
                        credentialsId: 'sonar-token',
```

```

        variable: 'SONAR_TOKEN'
    )
D) {
    sh """
        ${SONAR_SCANNER} \
        -Dsonar.projectKey=${SONAR_PROJECT_KEY} \
        -Dsonar.projectName=${APP_NAME} \
        -Dsonar.sources=src \
        -Dsonar.host.url=${SONAR_HOST} \
        -Dsonar.token=${SONAR_TOKEN}
    """
}

    echo "SonarQube analysis completed successfully"
}
}

// =====
// 3. MAVEN COMPILATION
// =====
stage('Maven Compilation') {
    steps {
        script {
            def mvnHome = tool 'Maven'

            echo "Using Maven from: ${mvnHome}"

            sh """
                ${mvnHome}/bin/mvn clean package -DskipTests
            """
        }
    }
}

// =====
// 4. GET MAVEN VERSION
// =====
stage('Get Version') {
    steps {
        script {
            def mvnHome = tool 'Maven'

            env.APP_VERSION = sh(
                script: """
                    ${mvnHome}/bin/mvn \
                    help:evaluate \
                    -Dexpression=project.version \
                    -q \
                    -DforceStdout
                """,
                returnStdout: true
            ).trim()

            echo "Application Version: ${env.APP_VERSION}"
        }
    }
}
}

```

```

}

// =====
// 5. NEXUS ARTIFACTORY
// =====
stage('Nexus Artifactory') {
    steps {
        script {
            def warFile = "target/hiring.war"

            sh """
                echo "Application Version: ${env.APP_VERSION}"
                ls -lh target/
                test -f "${warFile}"
            """

            nexusArtifactUploader(
                nexusVersion: 'nexus3',
                protocol: 'http',
                nexusUrl: NEXUS_URL,
                groupId: 'in.javahome',
                version: env.APP_VERSION,
                repository: NEXUS_REPOSITORY,
                credentialsId: 'nexus-credentials',
                artifacts: [[
                    artifactId: 'hiring',
                    classifier: "",
                    file: warFile,
                    type: 'war'
                ]]
            )

            echo "Artifact uploaded to Nexus successfully"
        }
    }
}

// =====
// 6. SLACK SUCCESS NOTIFICATION
// =====
stage('Slack Notification') {
    steps {
        slackSend(
            channel: 'C0BQWR8R0QG',
            color: 'good',
            message: """
BUILD SUCCESS

Application : ${APP_NAME}
Branch      : ${BRANCH}
Version     : ${env.APP_VERSION}
Build       : #${env.BUILD_NUMBER}

Nexus upload completed successfully.
            """
        )
    }
}

```

```

    }
}

// =====
// 7. DEPLOY ON TOMCAT
// =====
stage('Deploy On Tomcat') {
    steps {
        script {
            def warFile = sh(
                script: '''
                    find target -maxdepth 1 -type f -name "*.war" | head -1
                ''',
                returnStdout: true
            ).trim()

            if (!warFile) {
                error "No WAR file found in Jenkins workspace"
            }

            echo "Deploying WAR: ${warFile}"

            withCredentials([
                sshUserPrivateKey(
                    credentialsId: 'nexus-tomcat-ssh',
                    keyFileVariable: 'SSH_KEY',
                    usernameVariable: 'SSH_USER'
                )
            ]) {
                sh '''
                    echo "Copying WAR to Tomcat server..."

                    scp -i "${SSH_KEY}" \
                        -o StrictHostKeyChecking=no \
                        "${warFile}" \
                        ${SSH_USER}@${TOMCAT_HOST}:/tmp/${APP_NAME}.war

                    ssh -i "${SSH_KEY}" \
                        -o StrictHostKeyChecking=no \
                        ${SSH_USER}@${TOMCAT_HOST} '
                        sudo rm -rf ${TOMCAT_DEPLOY_PATH}/${APP_NAME}
                        sudo rm -f ${TOMCAT_DEPLOY_PATH}/${APP_NAME}.war

                        sudo cp /tmp/${APP_NAME}.war \
                            ${TOMCAT_DEPLOY_PATH}/${APP_NAME}.war

                        sudo chown tomcat:tomcat \
                            ${TOMCAT_DEPLOY_PATH}/${APP_NAME}.war

                        sudo systemctl restart tomcat
                        sudo systemctl is-active tomcat
                    '

                '''
            }

            echo "Application deployed successfully on Tomcat"
        }
    }
}

```

```

    }
  }
}

post {
    failure {
        script {
            echo "Pipeline failed"

            slackSend(
                channel: 'C0BQWR8R0QG',
                color: 'danger',
                message: """"
BUILD FAILED

Application : ${APP_NAME}
Branch      : ${BRANCH}
Build       : #${env.BUILD_NUMBER}

Check Jenkins console output for details.
        """"
    )
  }
}

success {
    echo "Pipeline completed successfully"
}
}
}

```

 **Screenshot Evidence: capture the console view for this step and insert it here.**

```

timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 9405b107fdffdf68e69fe953ea1f9669784f9d6a (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 9405b107fdffdf68e69fe953ea1f9669784f9d6a # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git checkout -b main 9405b107fdffdf68e69fe953ea1f9669784f9d6a # timeout=10
Commit message: "Update README.md"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // script
[Pipeline] echo
Git clone completed successfully

```

```
13:49:38.489 INFO Analysis report compressed in 20ms, zip size=31.5 kB
13:49:38.509 INFO Analysis report uploaded in 22ms
13:49:38.509 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://34.235.166.114:9000/dashboard?id=hrringapp
13:49:38.509 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
13:49:38.509 INFO More about the report processing at http://34.235.166.114:9000/api/ce/task?id=81f1b56f-d75d-4c88-a277-c1434be25245
13:49:38.512 INFO Analysis total time: 3.670 s
13:49:38.513 INFO SonarScanner Engine completed successfully
13:49:38.536 INFO EXECUTION SUCCESS
13:49:38.537 INFO Total time: 11.229s
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] echo
SonarQube analysis completed successfully
```

The screenshot shows the SonarQube web interface. The top navigation bar includes 'Projects', 'Issues', 'Rules', 'Quality profiles', 'Quality gates', 'Administration', and 'More'. The left sidebar has 'My Favorites' and 'All' tabs, and a 'Filters' section with 'Quality gate' (3 Passed, 1 Failed) and 'Security' (1 info issue). The main content area displays the 'hrringapp' project, which is 'Public' and 'Passed'. It shows the last analysis was 6 minutes ago with 11 Lines of Code (XML, JSP). The project's quality metrics are: Security (A 0), Reliability (C 3), Maintainability (A 0), Hotspots Reviewed (A —), Coverage (—), and Duplications (0.0%).

```
[INFO] No sources to compile
[INFO]
[INFO] --- surefire:3.5.4:test (default-test) @ hiring ---
[INFO] Tests are skipped.
[INFO]
[INFO] --- war:3.3.1:war (default-war) @ hiring ---
[INFO] Packaging webapp
[INFO] Assembling webapp [hiring] in [/var/lib/jenkins/workspace/hiring-app-multistage/target/hiring]
[INFO] Processing war project
[INFO] Copying webapp resources [/var/lib/jenkins/workspace/hiring-app-multistage/src/main/webapp]
[INFO] Building war: /var/lib/jenkins/workspace/hiring-app-multistage/target/hiring.war
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.347 s
[INFO] Finished at: 2026-08-20T13:49:42Z
[INFO] -----
```

```
drwxr-xr-x. 2 jenkins jenkins   28 Aug 20 13:49 maven-archiver
+ test -f target/hiring.war
[Pipeline] nexusArtifactUploader
Uploading artifact hiring.war started...
GroupId: in.javahome
ArtifactId: hiring
Classifier:
Type: war
Version: 0.1
File: hiring.war
Repository:hiringapp
Uploading: http://35.175.205.126:8081/repository/hiringapp/in/javahome/hiring/0.1/hiring-0.1.war
100 % completed (1.7 kB / 1.7 kB).
Uploaded: http://35.175.205.126:8081/repository/hiringapp/in/javahome/hiring/0.1/hiring-0.1.war (1.7 kB at 39 kB/s)
Uploading artifact hiring.war completed.
[Pipeline] echo
Artifact uploaded to Nexus successfully
```

```
active
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] echo
Application deployed successfully on Tomcat
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline completed successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



The screenshot shows the Tomcat Manager interface at 35.175.205.126:8080/manager/html. The 'Applications' table lists several web applications, with **/hiringapp** highlighted in green and a red border, indicating it is running successfully. Below the table, the 'Deploy' section shows the application is deployed from a directory.

Applications Table:

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/hiringapp	None specified	Archetype Created Web Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

The browser address bar shows the URL `35.175.205.126:8080/hiringapp/` and the page content displays 'Welcome to Techie Horizon'.

Step 4: Run the Job and Review Results

Click Build Now, then open Console Output to watch all six stages run in order, and confirm the image appears in the Docker Hub repository tagged with the build number.

Screenshot Evidence: capture the console view for this step and insert it here.

The screenshot shows the Jenkins Pipeline View for the 'hiring-app-multistage' job. The pipeline is in a successful state, and the 'Stage View' displays the following stages and their durations:

Stage	Git Clone	SonarQube Integration	Maven Compilation	Get Version	Nexus Artifacts	Slack Notification	Deploy On Tomcat	Declarative: Post Actions
Average stage times: (full run time: ~21s)	385ms	12s	3s	2s	434ms	156ms	1s	54ms
#2 Aug 20 19:19 No Changes	385ms	12s	3s	2s	434ms	156ms	1s	54ms

The 'Builds' section shows the latest build (#2) is successful, with a duration of 2 min 30 sec ago. The 'Permalinks' section provides links to various build details.

Explanation

This pipeline chains six stages end to end so that each one only starts after the previous one succeeds: source retrieval, compile, unit test, package, image build, and image push. The environment block computes an

IMAGE_TAG from the Jenkins build number once, and every later stage reuses it so the same tag is used to build, tag, and push the image. The withCredentials wrapper injects the Docker Hub username and password into the shell environment only for the Docker Push stage, keeping the token out of the rest of the pipeline and out of Console Output.

Real-Time Use Case

This is the shape of a typical containerized delivery pipeline: a team building a Java web application packages it as a WAR, bakes that WAR into a Docker image using the project's own Dockerfile, and pushes a uniquely tagged image on every build so a deployment pipeline (or ArgoCD, as referenced in the repository's own Jenkinsfile) can pick up the new tag and roll it out.

Common Errors and Troubleshooting

Error	Cause	Resolution
permission denied while trying to connect to the Docker daemon socket	Jenkins service account is not in the docker group on the agent	Add the jenkins user to the docker group on the agent and restart the Jenkins service
denied: requested access to the resource is denied (docker push)	Wrong or expired docker-hub credential, or image not tagged with the account namespace	Re-check the docker-hub credential and confirm the image is tagged as <dockerhub-username>/hiring-app:<tag>
Docker Build stage cannot find Dockerfile	Working directory does not contain the Dockerfile at build time	Confirm the Git Clone stage checked out the repository root, where the Dockerfile already exists

Task 4: Parallel-Stage Pipeline Job (hiring-app-parallel)

Outcome

A fourth Jenkins pipeline job named hiring-app-parallel runs against the hiring-app repository with a Git Clone stage, a Build stage, a Parallel Checks stage that runs Unit Tests, Static Code Analysis, and Dependency Check at the same time, and a final Archive Artifact stage.

Objective

To reduce total pipeline run time by running independent verification steps concurrently instead of one after another, using the Declarative parallel directive so that Unit Tests, Static Code Analysis, and a Dependency Check all execute on separate branches of the same stage.

Prerequisites

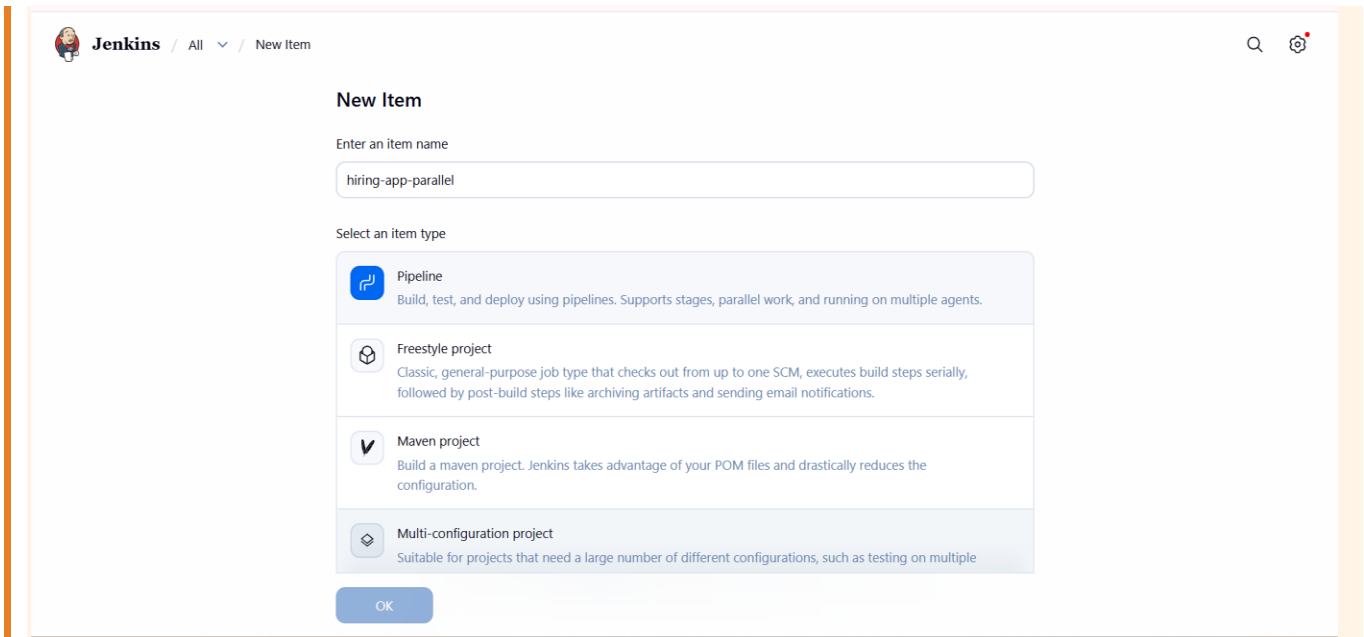
- Jenkins master with the Pipeline and Git plugins installed.
- The maven-3.9 Maven installation already registered from Task 1.
- Enough Jenkins executors available on the agent to run three branches of the parallel stage at the same time.
- Public read access to <https://github.com/betawins/hiring-app.git>.

Step-by-Step Implementation

Step 1: Create the Pipeline Job

Parameter	Value
Location	New Item -> Pipeline
Item name	hiring-app-parallel
Definition	Pipeline script

 **Screenshot Evidence: capture the console view for this step and insert it here.**



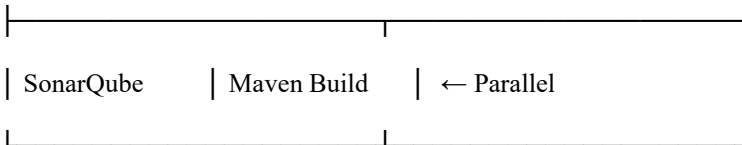
Step 2: Add the Parallel-Stage Jenkinsfile

Git Clone

|



BUILD & ANALYSIS



|

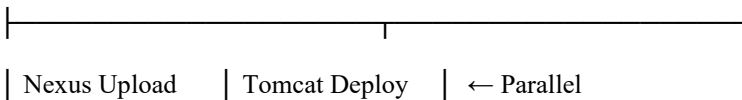


Get Version

|



RELEASE



Slack Notification

Use the following Jenkinsfile. The Parallel Checks stage declares three named branches inside a single parallel { } block; Jenkins schedules all three to start as soon as the Build stage finishes.

Jenkinsfile

```
pipeline {
    agent any

    environment {
        APP_NAME      = "hiringapp"
        ARTIFACT_NAME  = "HrringApp"
        SONAR_PROJECT_KEY = "hrringapp"

        GIT_REPO      = "https://github.com/betawins/hiring-app.git"
        BRANCH         = "main"

        SONAR_HOST     = "http://34.235.166.114:9000/"
        SONAR_SCANNER  = "/opt/sonar_scanner/bin/sonar-scanner"

        NEXUS_URL      = "35.175.205.126:8081"
        NEXUS_REPOSITORY = "hiringapp"

        TOMCAT_HOST    = "35.175.205.126"
        TOMCAT_DEPLOY_PATH = "/opt/tomcat/webapps"
    }

    stages {
        // =====
        // 1. GIT CLONE
        // =====
        stage('Git Clone') {
            steps {
                echo "Cloning GitHub Repository..."

                git branch: "${BRANCH}",
                    url: "${GIT_REPO}"

                echo "Git clone completed successfully"
            }
        }

        // =====
        // 2. SONARQUBE + MAVEN IN PARALLEL
        // =====
        stage('Build & Analysis - Parallel') {
            parallel {
```

```

// -----
// SONARQUBE
// -----
stage('SonarQube Integration') {
    steps {
        echo "Starting SonarQube analysis..."

        withCredentials([
            string(
                credentialsId: 'sonar-token',
                variable: 'SONAR_TOKEN'
            )
        ]) {
            sh """
                ${SONAR_SCANNER} \
                -Dsonar.projectKey=${SONAR_PROJECT_KEY} \
                -Dsonar.projectName=${APP_NAME} \
                -Dsonar.sources=src \
                -Dsonar.host.url=${SONAR_HOST} \
                -Dsonar.token=${SONAR_TOKEN}
            """
        }

        echo "SonarQube analysis completed successfully"
    }
}

// -----
// MAVEN BUILD
// -----
stage('Maven Compilation') {
    steps {
        script {
            def mvnHome = tool 'Maven'

            echo "Using Maven from: ${mvnHome}"

            sh """
                ${mvnHome}/bin/mvn clean package -DskipTests
            """
        }

        echo "Maven build completed successfully"
    }
}

}

// =====
// 3. GET APPLICATION VERSION
// =====
stage('Get Version') {
    steps {
        script {
            def mvnHome = tool 'Maven'

```

```

env.APP_VERSION = sh(
  script: ""
    ${mvnHome}/bin/mvn \
    help:evaluate \
    -Dexpression=project.version \
    -q \
    -DforceStdout
  "",
  returnStdout: true
).trim()

echo "Application Version: ${env.APP_VERSION}"
}
}
}

// =====
// 4. NEXUS + TOMCAT IN PARALLEL
// =====
stage('Release - Parallel') {

  parallel {

    // -----
    // NEXUS UPLOAD
    // -----
    stage('Upload To Nexus') {
      steps {
        script {
          def warFile = "target/hiring.war"

          sh ""
            echo "=====
            echo "Uploading artifact to Nexus"
            echo "Version: ${env.APP_VERSION}"
            echo "=====

          ls -lh target/
          test -f "${warFile}"
          ""

          nexusArtifactUploader(
            nexusVersion: 'nexus3',
            protocol: 'http',
            nexusUrl: NEXUS_URL,
            groupId: 'in.javahome',
            version: env.APP_VERSION,
            repository: NEXUS_REPOSITORY,
            credentialsId: 'nexus-credentials',
            artifacts: [[
              artifactId: 'hiring',
              classifier: "",
              file: warFile,
              type: 'war'
            ]]
          )
        }
      }
    }
  }
}

```

```

    echo "Artifact uploaded to Nexus successfully"
  }
}

// -----
// TOMCAT DEPLOYMENT
// -----

stage('Deploy On Tomcat') {
  steps {
    script {
      def warFile = "target/hiring.war"

      sh """
        echo "=====
        echo "Deploying application to Tomcat"
        echo "WAR File: ${warFile}"
        echo "=====

        test -f "${warFile}"
      """

      withCredentials([
        sshUserPrivateKey(
          credentialsId: 'nexus-tomcat-ssh',
          keyFileVariable: 'SSH_KEY',
          usernameVariable: 'SSH_USER'
        )
      ]) {
        sh """
          echo "Copying WAR to Tomcat server..."

          scp -i "${SSH_KEY}" \
            -o StrictHostKeyChecking=no \
            "${warFile}" \
            ${SSH_USER}@${TOMCAT_HOST}:/tmp/${APP_NAME}.war

          ssh -i "${SSH_KEY}" \
            -o StrictHostKeyChecking=no \
            ${SSH_USER}@${TOMCAT_HOST} '
            echo "Removing old deployment..."

            sudo rm -rf ${TOMCAT_DEPLOY_PATH}/${APP_NAME}
            sudo rm -f ${TOMCAT_DEPLOY_PATH}/${APP_NAME}.war

            echo "Deploying new WAR..."

            sudo cp /tmp/${APP_NAME}.war \
              ${TOMCAT_DEPLOY_PATH}/${APP_NAME}.war

            sudo chown tomcat:tomcat \
              ${TOMCAT_DEPLOY_PATH}/${APP_NAME}.war

            echo "Restarting Tomcat..."

            sudo systemctl restart tomcat
            sudo systemctl is-active tomcat
          '
        """
      }
    }
  }
}

```

```

        }

        echo "Application deployed successfully on Tomcat"
    }
}
}

// =====
// 5. SLACK SUCCESS NOTIFICATION
// =====
stage('Slack Notification') {
    steps {
        slackSend(
            channel: 'C0BQWR8R0QG',
            color: 'good',
            message: """"
BUILD SUCCESS

Application : ${APP_NAME}
Branch      : ${BRANCH}
Version     : ${env.APP_VERSION}
Build       : #${env.BUILD_NUMBER}

SonarQube   : SUCCESS
Maven       : SUCCESS
Nexus       : SUCCESS
Tomcat      : SUCCESS
        """"
    )
}
}

// =====
// POST BUILD ACTIONS
// =====
post {

    failure {
        script {
            echo "Pipeline failed"

            slackSend(
                channel: 'C0BQWR8R0QG',
                color: 'danger',
                message: """"
BUILD FAILED

Application : ${APP_NAME}
Branch      : ${BRANCH}
Build       : #${env.BUILD_NUMBER}

```


Check Jenkins console output for details.

```
        ""
    )
}
}

success {
    echo "Pipeline completed successfully"
}
}
```

 **Screenshot Evidence: capture the console view for this step and insert it here.**

```
timeout=10
> git config remote.origin.url https://github.com/betawins/hiring-app.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 9405b107fdffdf68e69fe953ea1f9669784f9d6a (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 9405b107fdffdf68e69fe953ea1f9669784f9d6a # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git checkout -b main 9405b107fdffdf68e69fe953ea1f9669784f9d6a # timeout=10
Commit message: "Update README.md"
First time build. Skipping changelog.
[Pipeline] echo
Git clone completed successfully
```

```
[Pipeline] echo
[Pipeline] script
[Pipeline] {
[Pipeline] tool
[Pipeline] {
[Pipeline] sh
[Pipeline] echo
[Maven Compilation] Using Maven from: /var/lib/jenkins/tools/hudson.tasks.Maven_MavenInstallation/Maven
[Pipeline] sh
[SonarQube Integration] + /opt/sonar_scanner/bin/sonar-scanner -Dsonar.projectKey=hrringapp -Dsonar.projectName=hrringapp -Dsonar.sources=src -
Dsonar.host.url=http://34.235.166.114:9000/ -Dsonar.token=****
[Maven Compilation] + /var/lib/jenkins/tools/hudson.tasks.Maven_MavenInstallation/Maven/bin/mvn clean package -DskipTests
[SonarQube Integration] 14:08:03.699 INFO Scanner configuration file: /opt/sonar_scanner/conf/sonar-scanner.properties
14:08:03.706 INFO Project root configuration file: NONE
14:08:03.736 INFO SonarScanner CLI 8.1.0.6389
14:08:03.742 INFO Linux 6.18.39-79.141.amzn2023.x86_64 amd64
[Maven Compilation] [INFO] Scanning for projects...
[INFO]
[INFO] -----< in.javahome:hiring >-----
[INFO] Building hiring Maven Webapp 0.1
[INFO] from pom.xml
```

```
[INFO] Finished at: 2026-08-20T14:08:08Z
[INFO] -----
[Pipeline] }
[Pipeline] // script
[Pipeline] echo
[Maven Compilation] Maven build completed successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[SonarQube Integration] 14:08:09.962 INFO    Communicating with SonarQube Community Build 26.8.0.126808
14:08:09.963 INFO    JRE provisioning: os[linux], arch[x86_64]
14:08:10.425 INFO    Starting SonarScanner Engine...
14:08:10.434 INFO    Java 21.0.11 Eclipse Adoptium (64-bit)
14:08:14.284 INFO    Load global settings
14:08:14.383 INFO    Load global settings (done) | time=96ms
14:08:14.387 INFO    Server id: 147B411E-AaAVvibzAAV4sS23tiSz
14:08:14.402 INFO    Loading required plugins
14:08:14.405 INFO    Load plugins index
14:08:14.424 INFO    Load plugins index (done) | time=17ms
14:08:14.425 INFO    Load/download plugins
14:08:14.454 INFO    Load/download plugins (done) | time=34ms
```

```
14:08:18.754 INFO    EXECUTION SUCCESS
14:08:18.754 INFO    Total time: 15.062s
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] echo
[SonarQube Integration] SonarQube analysis completed successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // parallel
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Get Version) \(hide\)
[Pipeline] script
[Pipeline] {
[Pipeline] tool
[Pipeline] sh
```

```
ing-app-parallel/1/console# + /var/lib/jenkins/tools/hudson.tasks.Maven_MavenInstallation/Maven/bin/mvn help:evaluate -Dexpression=project.version -q -
```

```
[Upload To Nexus] + test -f target/hiring.war
[Pipeline] nexusArtifactUploader
Uploading artifact hiring.war started....
GroupId: in.javahome
ArtifactId: hiring
Classifier:
Type: war
Version: 0.1
File: hiring.war
Repository:hiringapp
Uploading: http://35.175.205.126:8081/repository/hiringapp/in/javahome/hiring/0.1/hiring-0.1.war
[Deploy On Tomcat] + echo =====
=====
+ echo 'Deploying application to Tomcat'
Deploying application to Tomcat
+ echo 'WAR File: target/hiring.war'
WAR File: target/hiring.war
+ echo =====
=====
+ test -f target/hiring.war
[Upload To Nexus] 100 % completed (1.7 kB / 1.7 kB).
[Pipeline] withCredentials
[Deploy On Tomcat] Making suggested pattern matches of $SSH_KEY
```

```

[Pipeline] // parallel
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Slack Notification)
[Pipeline] slackSend
Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: khadeer, channel: C0BQWR8R0QG, color: good,
botUser: true, tokenCredentialId: slack, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline completed successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

The screenshot shows the Sonatype Nexus Repository Community Edition web interface. The top navigation bar includes a search bar and user profile icons. The left sidebar contains links for Dashboard, Search, Browse (active), Upload, and Settings. The main content area displays the 'Browse' view for the 'hiringapp' component. It shows a tree structure with folders 'in', 'javahome', and 'hiring'. Under 'hiring', there is a folder '01' containing three files: 'hiring-01.war', 'hiring-01.war.md5', and 'hiring-01.war.sha1'.

The screenshot shows a Slack workspace interface. The left sidebar displays the user profile 'khadeer' and a list of channels, including '#jenkins_notification' which is currently selected. The main window shows a message from the 'Jenkins' app, timestamped '7:38 PM'. The message content is as follows:

```

BUILD SUCCESS
Application : hiringapp
Branch      : main
Version     : 0.1
Build       : #1

SonarQube   : SUCCESS
Maven       : SUCCESS
Nexus       : SUCCESS
Tomcat      : SUCCESS

```

Below the message is a text input field with a rich text editor toolbar (bold, italic, underline, link, etc.) and a 'Send' button.

[←](#)
[→](#)
[↻](#)
Not secure
35.175.205.126:8080/manager/html

message:

Manager

[List Applications](#)
[HTML Manager Help](#)
[Manager Help](#)
[Server Status](#)

Applications

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/hiringapp	None specified	Archetype Created Web Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

Deploy

Deploy directory or WAR file located on server

←

→

↻

Not secure
35.175.205.126:8080/hiringapp/

Welcome to Techie Horizon

Step 3: Run the Job and Review Results

Click Build Now, then open the Stage View and confirm the three Parallel Checks branches start together and run side by side rather than one after another.

📷

Screenshot Evidence: capture the console view for this step and insert it here.

Jenkins

hiring-app-parallel

Status

Changes

Build Now

Delete Pipeline

Configure

Rename

Full Stage View

Stages

Pipeline Syntax

Builds >

Today

#1 2:08 PM

hiring-app-parallel

verage stage times:

full run time: ~22s

Git Clone	Build & Analysis - Parallel	SonarQube Integration	Maven Compilation	Get Version	Release - Parallel	Upload To Nexus	Deploy On Tomcat	Slack Notification	Declarative: Post Actions
835ms	76ms	16s	6s	2s	64ms	555ms	1s	163ms	61ms
835ms	76ms	16s	6s	2s	64ms	555ms	1s	163ms	61ms

Permalinks

REST API

Jenkins 2.568.2

Validation Steps

- Console Output interleaves log lines from Unit Tests, Static Code Analysis, and Dependency Check, showing they are running concurrently.

- The Stage View displays the Parallel Checks stage as three side-by-side branches rather than a single linear box.
- Total pipeline duration for the Parallel Checks stage is close to the slowest individual branch, not the sum of all three.
- If any one parallel branch fails, the Parallel Checks stage is marked failed and the Archive Artifact stage is skipped.

Explanation

The parallel directive inside a Declarative stage lets Jenkins allocate an executor to each named branch and run them concurrently instead of Jenkins waiting for one steps block to finish before starting the next. Here, Unit Tests, Static Code Analysis, and Dependency Check do not depend on each other's output, so they are safe to run at the same time; only the Archive Artifact stage after them needs to wait for all three branches to complete.

Real-Time Use Case

Once a codebase has several independent verification steps — unit tests, linting or static analysis, dependency or vulnerability scanning — running them in parallel is a common way to keep pipeline duration flat as more checks are added, instead of every new check tacking its full runtime onto the end of the pipeline.

Common Errors and Troubleshooting

Error	Cause	Resolution
Parallel branches run one after another instead of together	Agent only has a single executor available, so branches queue instead of running concurrently	Increase the number of executors on the agent, or configure agent {} per branch so each runs on its own node
One failed branch does not stop the others	This is expected Declarative behavior; parallel branches run to completion independently by default	Add failFast true inside the parallel block if the other branches should be aborted as soon as one fails
checkstyle:check fails with 'Plugin not found'	The maven-checkstyle-plugin is not declared in pom.xml	Add the maven-checkstyle-plugin to the build/plugins section of pom.xml before running this stage